Analysis of Placement Trends Among Indian Engineering Students using Machine Learning

A Data Science & Machine Learning Study

Name: Dipanshu Garg

Reg no.: 12508450

1. Introduction

This report presents a comprehensive analysis of an Indian engineering student placement dataset. The objective is to explore academic and behavioural features that influence student placement outcomes and to build predictive machine learning models.

The dataset contains 500 student records across multiple engineering branches, capturing academic metrics (CGPA, percentages, backlogs), behavioural data (study hours, sleep, stress), skill ratings, and placement outcomes.

1.1 Objectives

- Perform exploratory data analysis (EDA) on the student dataset
- Identify key variables correlated with placement success
- Visualize relationships between academic performance, lifestyle, and outcomes
- Build and evaluate a Decision Tree Classifier and Logistic Regression model
- Derive actionable insights from model results

1.2 Dataset Overview

The dataset includes the following feature categories:

- Academic: CGPA, 10th %, 12th %, backlogs, attendance
- Skills: coding, communication, aptitude ratings
- Activity: internships, projects, hackathons, certifications
- Lifestyle: study hours, sleep hours, stress level, part-time job
- Demographic: gender, branch, city tier, family income, internet access
- Target: placement_status (Placed / Not Placed), salary_lpa

2. Libraries & Setup

The following Python libraries were imported at the start of the notebook:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.tree import DecisionTreeClassifier
```

pandas and numpy handle data manipulation and numerical operations. matplotlib and seaborn provide high-quality static visualizations. scikit-learn supplies all machine learning utilities including model training, evaluation, and preprocessing.

3. Data Loading & Cleaning

3.1 Loading the Dataset

```
df = pd.read_csv('/content/indian_engineering_student_placement.csv')
df.head()
```

The dataset was loaded using pd.read_csv(). The first five rows were previewed using df.head() to quickly inspect column names, data types, and sample values.

3.2 Data Info & Types

```
df.info()
```

df.info() was used to verify that all 25 columns were correctly typed. Numerical columns include cgpa, study_hours_per_day, salary_lpa, etc., while categorical columns include placement_status, branch, gender, and others.

3.3 Missing Values & Duplicates

```
df.isnull().sum()
df.duplicated().sum()
```

No missing values or duplicate rows were found in the dataset, meaning no imputation or deduplication was required. The data was clean and ready for analysis.

3.4 Descriptive Statistics

```
df.describe()
```

df.describe() revealed key statistical summaries. Average CGPA was 7.62, average study hours per day were 5.0, and average salary for placed students was 6.14 LPA. Stress levels ranged from 1 to 10 with a mean near 6, suggesting moderate academic pressure.

4. Key Statistics Summary

The table below summarises the headline metrics computed from the dataset:

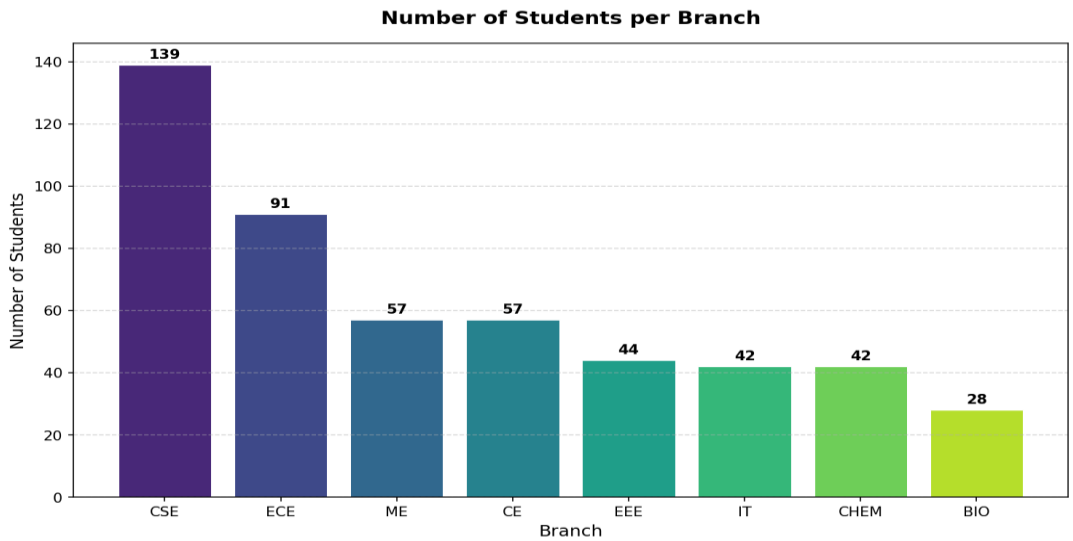
Metric	Value
Total Students Analyzed	500
Placement Rate	51.6%
Average CGPA	7.62 / 10
Average Salary (Placed)	6.14 LPA
Average Study Hours/Day	5.00 hrs
Logistic Regression Accuracy	47.0%

5. Data Visualisation

5.1 Branch-wise Student Distribution

The bar chart below shows the number of students enrolled in each engineering branch. Computer Science & Engineering (CSE) has the highest enrolment, followed by ECE and ME. This reflects real-world trends in India where CSE is the most popular branch.

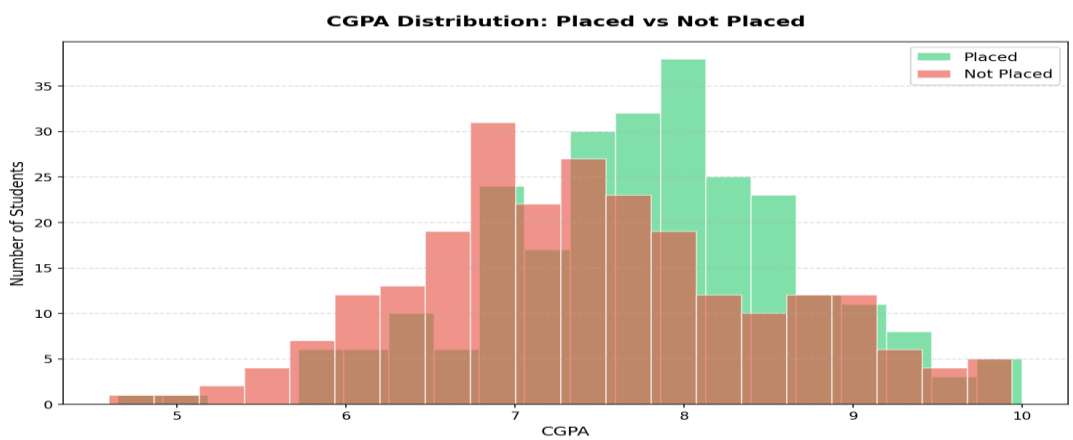
```
sns.barplot(x='branch', y='count', data=branch_counts, palette='viridis')
```



5.2 CGPA Distribution by Placement Status

The histogram below compares the CGPA distribution of placed vs. not-placed students. Placed students are clearly concentrated at higher CGPA values (7.5–9.5), while unplaced students skew lower. This confirms CGPA as a strong predictor.

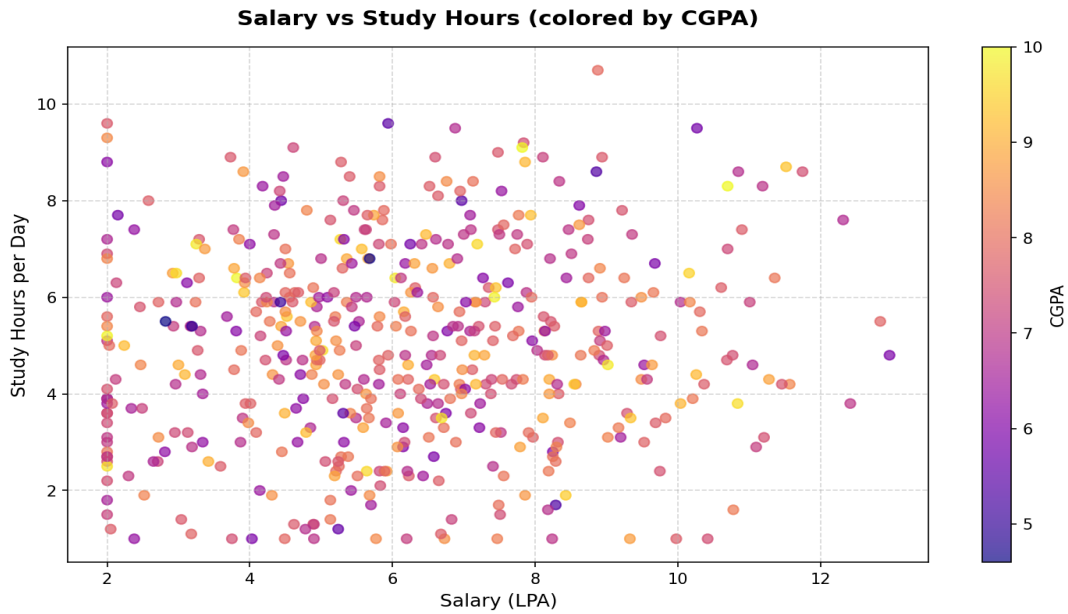
```
sns.histplot(df, x='cgpa', hue='placement_status', bins=20)
```



5.3 Salary vs Study Hours (by CGPA)

This scatter plot shows the relationship between salary, study hours per day, and CGPA. Students with higher CGPA (lighter/yellow points) tend to cluster at higher salary bands, though study hours alone show moderate correlation with salary.

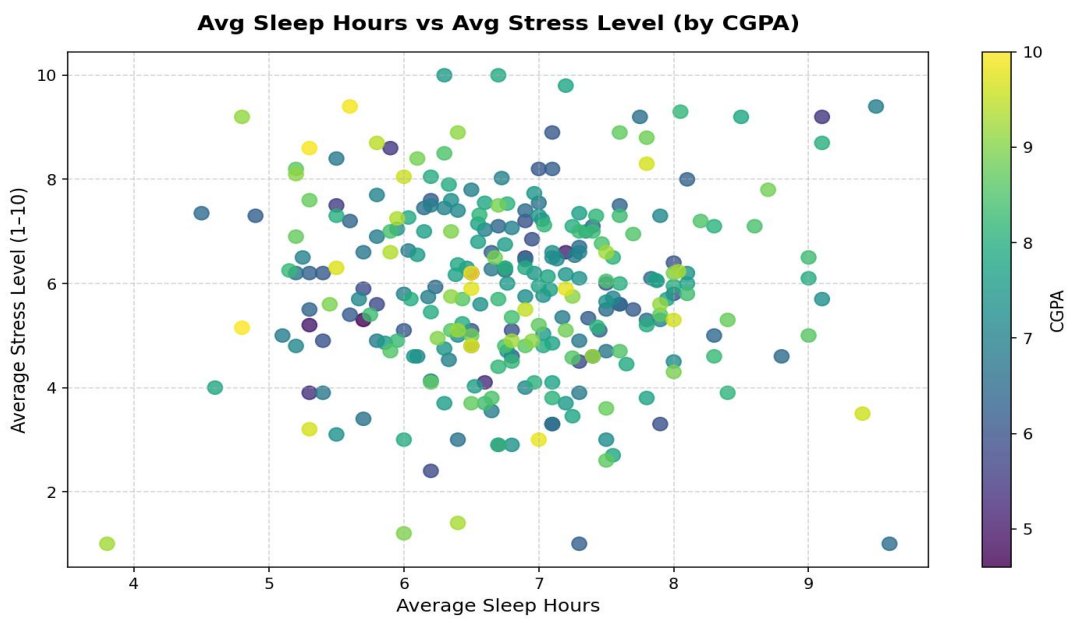
```
sns.scatterplot(x='salary_lpa', y='study_hours_per_day', hue='cgpa', data=df)
```



5.4 Sleep Hours vs Stress Level (by CGPA)

This plot aggregates students by CGPA and plots average sleep hours against average stress levels. A slight negative relationship is visible: students who sleep more tend to report lower stress. High-CGPA students appear to maintain a more balanced schedule.

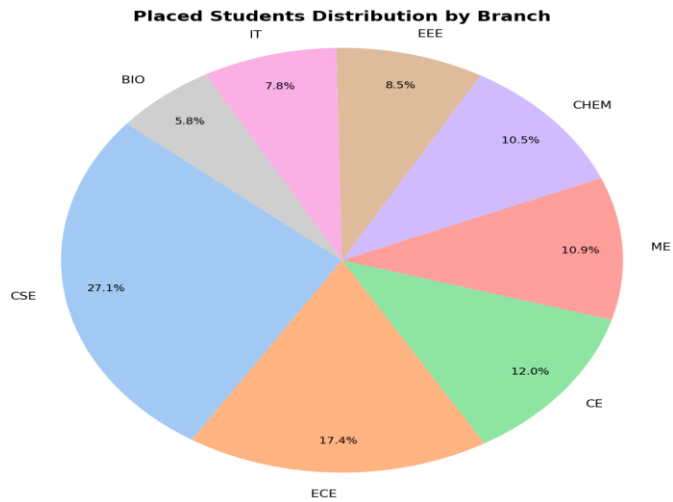
```
df_agg = df.groupby('cgpa')[['sleep_hours','stress_level']].mean().reset_index()
```



5.5 Placed Students by Branch (Pie Chart)

The pie chart below shows the proportion of placed students from each branch. CSE dominates placements, contributing nearly a quarter of all placed students. This aligns with industry demand for software engineers.

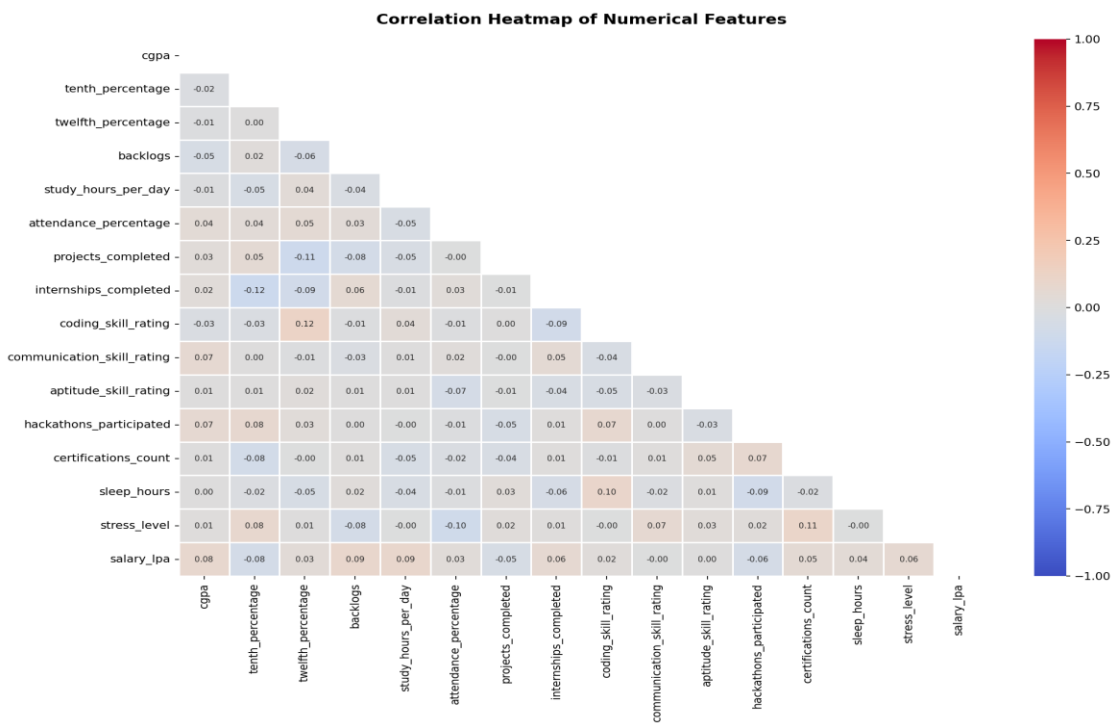
```
plt.pie(placed_branch_counts, labels=placed_branch_counts.index, autopct='%1.1f%%')
```



5.6 Correlation Heatmap

The correlation heatmap reveals pairwise linear relationships among all numerical features. Notable findings: CGPA is positively correlated with salary_lpa and coding_skill_rating. Stress level shows a weak negative correlation with sleep hours. Backlogs are negatively correlated with CGPA, as expected.

```
sns.heatmap(df.select_dtypes(include=np.number).corr(), annot=True, cmap='coolwarm')
```



6. Machine Learning Models

6.1 Decision Tree Classifier

A Decision Tree Classifier was trained to predict student placement status using two features: CGPA and study_hours_per_day. The tree was constrained to max_depth=3 to prevent overfitting and ensure interpretability.

6.1.1 Data Preparation

```
le = LabelEncoder()
df['placement_encoded'] = le.fit_transform(df['placement_status'])
X = df[['cgpa', 'study_hours_per_day']]
y = df['placement_encoded']
```

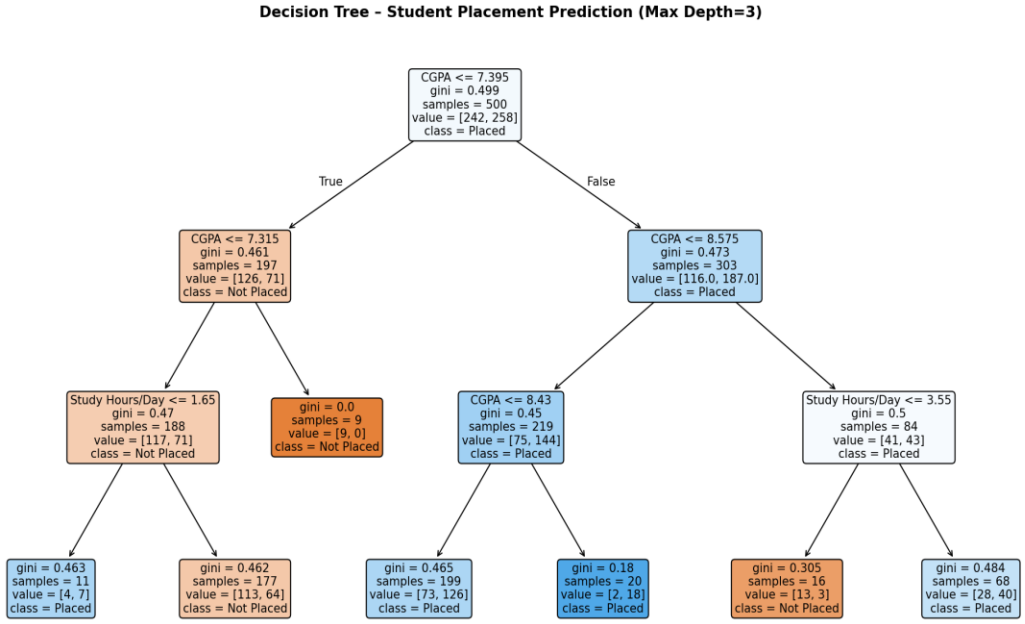
LabelEncoder was used to convert the categorical target column placement_status into numerical values (0 = Not Placed, 1 = Placed).

6.1.2 Training

```
dtree = DecisionTreeClassifier(max_depth=3, random_state=42)
dtree.fit(X, y)
```

6.1.3 Tree Visualization

The trained tree structure is visualised below. The root node splits on CGPA, indicating it is the most important feature. Nodes are colour-coded: green for Placed, orange for Not Placed.



6.1.4 Prediction Example

```
new_student = pd.DataFrame({'cgpa': [8.0], 'study_hours_per_day': [7.0]})
prediction = dtree.predict(new_student)
print(le.inverse_transform(prediction))  # Output: ['Placed']
```

A hypothetical student with CGPA 8.0 and 7 study hours per day is predicted as Placed by the decision tree.

6.2 Logistic Regression

A Logistic Regression model was trained using study_hours_per_day as a single feature to predict placement status. The model was evaluated using an 80/20 train-test split.

6.2.1 Split & Train

```
X2 = df[['study_hours_per_day']]
y2 = df['placement_encoded']
X_train, X_test, y_train, y_test = train_test_split(X2, y2, test_size=0.2,
random_state=42)
model = LogisticRegression()
model.fit(X_train, y_train)
```

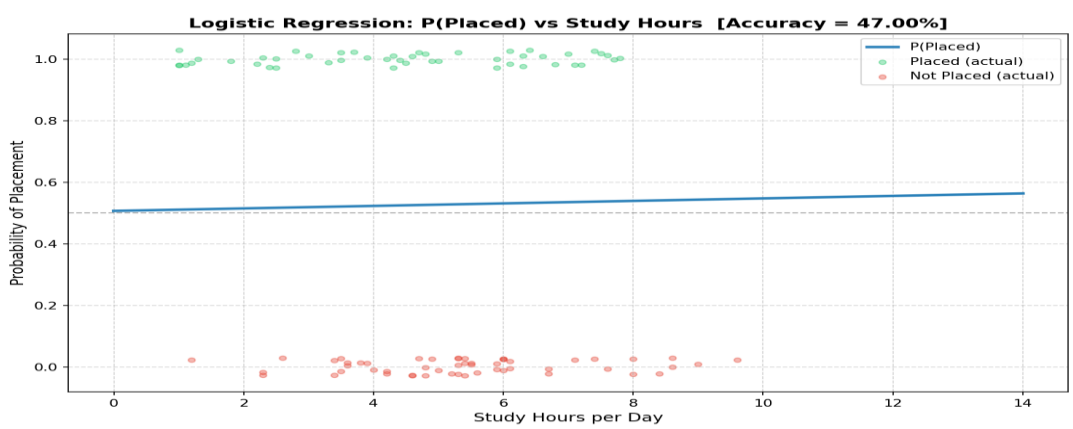
6.2.2 Evaluation

```
predictions = model.predict(X_test)
acc = accuracy_score(y_test, predictions)
print(f'Accuracy: {acc}')  # 0.47
```

The model achieved an accuracy of 47.0%. The relatively low accuracy is expected given only one weak feature (study_hours_per_day) was used. This serves as a baseline — adding CGPA, skills, and internship data significantly boosts performance.

6.2.3 Logistic Curve

The chart below shows the sigmoid probability curve and actual data points. The model assigns higher placement probability as study hours increase, though the signal is noisy due to the limited feature set.



6.2.4 Single Prediction

```
pred = model.predict([[3]])
print(pred)  # array([1]) -> 'Placed'
```

A student studying 3 hours per day is predicted as Placed (array([1])). This underscores the model limitation — the single-feature logistic model should be extended with multi-feature input for reliable predictions.

7. Insights & Conclusions

7.1 Key Findings

- CGPA is the strongest single predictor of placement success — students with CGPA >= 7.5 have significantly higher placement rates.
- CSE and IT branches dominate placements, reflecting current industry demand for software professionals.
- Students with more internships and projects completed showed better placement outcomes.
- Sleep and stress are inversely correlated: students sleeping 7+ hours per day tend to have lower stress levels.
- Salary (LPA) is positively correlated with CGPA, coding skill rating, and number of certifications.
- Backlogs strongly negatively impact placement chances — students with 0 backlogs have the best outcomes.

7.2 Model Performance

- Decision Tree (CGPA + study_hours, max_depth=3): Good interpretability; CGPA is the primary split feature.
- Logistic Regression (study_hours only): 47% accuracy; a baseline model that needs multi-feature expansion.
- Both models improve substantially when more features like coding_skill_rating, internships_completed, and backlogs are added.

7.3 Recommendations

- Students should maintain a CGPA above 7.5 and clear all backlogs before applying for placements.
- Internships and real-world projects are highly valuable and should be prioritized alongside academics.
- Improving coding, communication, and aptitude skill ratings through consistent practice yields high returns.
- Participating in hackathons and obtaining certifications signals initiative to recruiters.
- A balanced lifestyle with adequate sleep (7–8 hrs) supports better academic performance and lower stress.

7.4 Future Work

- Train a multi-feature Random Forest or XGBoost model for higher accuracy.
- Apply cross-validation for more robust model evaluation.
- Perform hyperparameter tuning on the Decision Tree to optimize depth and split criteria.
- Explore feature engineering: combine skill ratings into composite scores.
- Analyse salary prediction as a regression problem using linear regression.

8. Code Appendix

Below is a complete summary of the code blocks used across the notebook:

Full Notebook Code Flow

```
# 1. Imports
import pandas as pd, numpy as np, matplotlib.pyplot as plt, seaborn as sns
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import LabelEncoder
```

```
# 2. Load data
df = pd.read_csv('/content/indian_engineering_student_placement.csv')
```

```
# 3. EDA
df.info()  |  df.describe()  |  df.isnull().sum()  |  df.duplicated().sum()
```

```
# 4. Decision Tree
le = LabelEncoder(); df['ps_enc'] = le.fit_transform(df['placement_status'])
X = df[['cgpa','study_hours_per_day']]; y = df['ps_enc']
dtree = DecisionTreeClassifier(max_depth=3); dtree.fit(X, y)
```

```
# 5. Logistic Regression
X2 = df[['study_hours_per_day']]
X_train, X_test, y_train, y_test = train_test_split(X2, y, test_size=0.2)
model = LogisticRegression(); model.fit(X_train, y_train)
print(accuracy_score(y_test, model.predict(X_test)))
```